

W Y K Ł A D 3

Kolekcje w Pythonie

list, tuple, dict, set

Wprowadzenie do programowania II

Informatyka Stosowana | semestr letni 2025/2026

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- 1 list - lista: tworzenie, metody, slicing
- 2 List comprehension - złożenia listowe
- 3 Kopiowanie list: płytka vs głęboka kopia
- 4 tuple - krotka: niemutowalna sekwencja
- 5 dict - słownik: mapowanie klucz-wartość
- 6 set i frozenset - zbiory
- 7 Iteracja: range, enumerate, zip
- 8 Pułapki i idiomy kolekcji

list - lista

Mutowalny kontener sekwencyjny

Tworzenie listy

Cztery sposoby na stworzenie listy w Pythonie.

Sposoby tworzenia

```
# literal - elementy w nawiasach []
nums = [1, 2, 3, 4, 5]
mix = [1, "abc", True, 3.14]

# konstruktor list()
a = list("hello") # ['h', 'e', 'l', 'l', 'o']
b = list(range(5)) # [0, 1, 2, 3, 4]

# pusta lista
c = []
d = list()
```

Właściwości

✓ Mutowalna

Można zmieniać zawartość w miejscu

✓ Uporządkowana

Elementy zachowują kolejność dodawania

✓ Indeksowana

Dostęp po indeksie: `nums[0]`, `nums[-1]`

✓ Heterogeniczna

Elementy różnych typów w jednej liście

✓ Dynamiczna

Rozmiar zmienia się automatycznie

Metody listy

Przegląd najważniejszych metod modyfikujących listę.

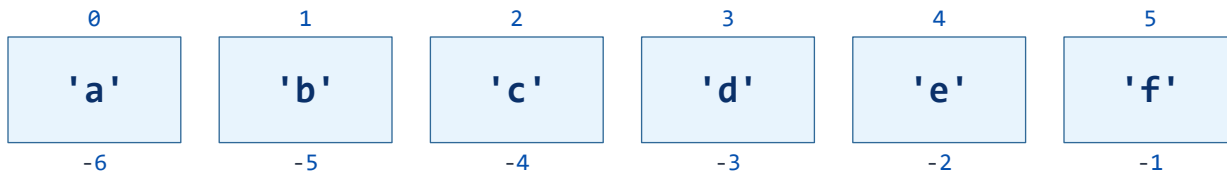
Metoda	Opis
<code>append(x)</code>	Dodaje x na koniec listy
<code>extend(iter)</code>	Dodaje elementy z iterowalnego obiektu
<code>insert(i, x)</code>	Wstawia x na pozycję i
<code>remove(x)</code>	Usuwa pierwsze wystąpienie x (ValueError jeśli brak)
<code>pop(i=-1)</code>	Usuwa i zwraca element na pozycji i
<code>clear()</code>	Usuwa wszystkie elementy
<code>sort(key, reverse)</code>	Sortuje listę w miejscu (in-place)
<code>reverse()</code>	Odwraca kolejność elementów w miejscu
<code>index(x)</code>	Zwraca indeks pierwszego wystąpienia x
<code>count(x)</code>	Zlicza wystąpienia x

⚠ Wszystkie powyższe metody (oprócz `index/count`) modyfikują listę in-place!

Slicing - wycinanie fragmentów

Slicing na listach działa tak samo jak na stringach, ale pozwala też modyfikować.

```
letters = ['a', 'b', 'c', 'd', 'e', 'f']
```



Odczyt (jak w str)

```
letters[1:4]    # ['b', 'c', 'd']  
letters[:3]     # ['a', 'b', 'c']  
letters[::2]    # ['a', 'c', 'e']  
letters[::-1]   # ['f', 'e', 'd', 'c', 'b', 'a']
```

Zapis (NOWOŚĆ vs str!)

```
letters[2:5] = ["X", "Y"] # podmiana  
# -> ['a', 'b', 'X', 'Y', 'f']  
letters[1:3] = [] # usunięcie  
# -> ['a', 'Y', 'f']  
letters[:] = [] # wyczyść całą
```

Lista jako stos i kolejka

Implementacja klasycznych struktur danych za pomocą listy.



Stos (LIFO)

```
stos = [1, 2, 3]

# push
stos.append(4)    # [1,2,3,4]

# pop
stos.pop()        # 4
                  # [1,2,3]

# peek
stos[-1]          # 3
```



Kolejka (FIFO)

```
kol = [1, 2, 3]

# enqueue
kol.append(4)     # [1,2,3,4]

# dequeue
kol.pop(0)        # 1
                  # [2,3,4]

# peek
kol[0]            # 2
```

Uwaga: pop(0) jest $O(n)$. Dla wydajnej kolejki - collections.deque

List comprehension - złożenia listowe

Zwięzły i idiomatyczny sposób tworzenia list w Pythonie.

[wyrażenie **for** zmienna **in** iterowalny **if** warunek]

Przykłady

```
# kwadraty liczb 0..9
[x**2 for x in range(10)]

# tylko parzyste
[x for x in range(20) if x%2==0]

# wielkie litery
[c.upper() for c in 'hello']
```

Zagnieżdżone + konwersja

```
# spłaszczenie macierzy 2D
matrix = [[1,2],[3,4],[5,6]]
[x for row in matrix for x in row]
# [1, 2, 3, 4, 5, 6]

# Fahrenheit -> Celsius
F_temps = [32, 68, 100, 212]
[round((f-32)*5/9,1) for f in F_temps]
```

Comprehension vs pętla for

Te same wyniki, różny styl - comprehension jest szybsze i bardziej pythonowe.

✗ Klasyczna pętla

```
wynik = []  
for x in range(10):  
    if x % 2 == 0:  
        wynik.append(x**2)  
# [0, 4, 16, 36, 64]
```

✓ List comprehension

```
wynik = [x**2 for x in range(10)  
         if x % 2 == 0]  
  
# [0, 4, 16, 36, 64]
```

Cecha	Pętla for	Comprehension
Czytelność	Dobra dla złożonych	Lepsza dla prostych
Wydajność	Wolniejsza (~20-30%)	Szybsza (C-loop)

Kopiowanie list - płytka vs głęboka

Przypisanie NIE kopiuje - to kluczowy punkt z wykładu 2.

✗ Przypisanie

```
a = [1, [2, 3]]
b = a
b[0] = 99

print(a)
# [99, [2, 3]]
# TEN SAM obiekt!
```

⚠ Płytka kopia

```
a = [1, [2, 3]]
b = a.copy()
b[0] = 99
print(a) # [1, [2, 3]]

b[1][0] = 99
print(a)
# [1, [99, 3]] !
```

✓ Głęboka kopia

```
import copy
a = [1, [2, 3]]
b = copy.deepcopy(a)
b[1][0] = 99

print(a)
# [1, [2, 3]]
# BEZPIECZNA!
```

Sposoby tworzenia płytkiej kopii: `a.copy()` | `list(a)` | `a[:]` | `copy.copy(a)`

⚠ Płytka kopia kopiuje referencje do zagnieżdżonych obiektów - nie same obiekty!

Tablice wielowymiarowe - lista list

Macierze w czystym Pythonie (bez NumPy).

Tworzenie macierzy 3x3

```
# OK - comprehension
matrix = [[0]*3 for _ in range(3)]
matrix[1][2] = 5
# [[0,0,0], [0,0,5], [0,0,0]]
```

❌ PUŁAPKA: mnożenie list

```
# ŹLE - 3 referencje do TEJ SAMEJ listy!
matrix = [[0]*3] * 3
matrix[1][2] = 5
# [[0,0,5], [0,0,5], [0,0,5]]
```

Sortowanie i przydatne funkcje

```
nums = [3, 1, 4, 1, 5]      sorted(nums) # [1,1,3,4,5] NOWA lista      nums.sort() # in-place
len(nums) # 5               sum(nums) # 14      min(nums) # 1      max(nums) # 5      any/all([True, False])
```

Rozpakowywanie (unpacking)

Elegancki sposób na przypisanie elementów kolekcji do zmiennych.

```
# Podstawowe rozpakowywanie
```

```
a, b, c = [1, 2, 3]          # a=1, b=2, c=3
```

```
# Rozpakowywanie z gwiazdką (*)
```

```
first, *rest = [1, 2, 3, 4, 5] # first=1, rest=[2,3,4,5]
```

```
*head, last = [1, 2, 3, 4, 5] # head=[1,2,3,4], last=5
```

```
a, *mid, z = [1, 2, 3, 4, 5]   # a=1, mid=[2,3,4], z=5
```

```
# Zamiana wartości (swap) - bez zmiennej tymczasowej!
```

```
x, y = 10, 20
```

```
x, y = y, x          # x=20, y=10
```

```
# Ignorowanie wartości
```

```
_, name, _ = ["id", "Alice", "extra"] # name="Alice"
```

tuple - krotka

Niemutowalna sekwencja

Tworzenie i właściwości krotki

Tworzenie

```
t1 = (1, 2, 3)      # z nawiasami
t2 = 1, 2, 3        # bez nawiasów
t3 = tuple("abcd")  # z iterable
t4 = tuple(range(5)) # z range
t5 = ()             # pusta krotka
```

UWAGA: jednoelementowa!

k = (1,) # krotka ✓

k = (1) # int! ✗

Właściwości

✓ Niemutowalna

Nie można zmieniać elementów

✓ Hashowalna*

Może być kluczem w dict / elementem set

✓ Szybsza niż list

Mniejszy narzut pamięciowy

✓ Bezpieczna

Gwarancja stałości danych

✓ Indeksowana

Dostęp i slicing jak w list

⚠ *Krotka jest hashowalna TYLKO gdy wszystkie jej elementy są hashowalne.

(1, [2, 3]) - **NIE** jest hashowalna (lista jest mutowalna).

Rozpakowywanie i namedtuple

Rozpakowywanie krotek

```
punkt = (3.5, 7.2)
x, y = punkt # x=3.5, y=7.2

# Funkcja zwracająca wiele wartości
def divmod_ex(a, b):
    return a // b, a % b
q, r = divmod_ex(17, 5) # q=3, r=2
```

collections.namedtuple

```
from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
p = Point(3, 7)
print(p.x)      # 3 (dostęp po nazwie)
print(p[0])     # 3 (dostęp po indeksie)
print(p)        # Point(x=3, y=7)
```

Kiedy używać tuple zamiast list?

- Dane stałe - współrzędne, kolory RGB, konfiguracja
- Klucze słownika - dict wymaga hashowalnych kluczy
- Zwracanie wielu wartości z funkcji
- Zabezpieczenie przed przypadkową modyfikacją

Mutowalny element w niemutowalnej krotce

Krotka jest niemutowalna, ale... jej mutowalny element można zmieniać.

```
t = ("abc", [1, 2, 3], True)

# Nie można zmienić elementu krotki:
t[0] = "xyz"      # TypeError: 'tuple' object does not support item assignment

# ALE można zmodyfikować ZAWARTOŚĆ mutowalnego elementu:
t[1].append(4)    # OK!
print(t)          # ("abc", [1, 2, 3, 4], True)

# Dlaczego? Krotka przechowuje REFERENCJĘ do listy.
# Referencja się nie zmienia - zmienia się zawartość listy.
```

 Taka krotka traci hashowalność - nie może być kluczem dict ani elementem set.

dict - słownik

Mapowanie klucz → wartość

Tworzenie słownika

Kilka sposobów na utworzenie obiektu dict.

```
# Literał z klamrami
d1 = {"a": "alpha", "b": "beta", "c": "gamma"}
# Konstruktor dict()
d2 = dict(name="Alice", age=30)
# Z listy krotek
d3 = dict([("x", 1), ("y", 2)])
# Dict comprehension
d4 = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}
# Pusty słownik
d5 = {} # LUB dict()
```

Właściwości

- ✓ Mutowalny
- ✓ Klucze unikalne
- ✓ Klucz = hashowalny
- ✓ Kolejność (3.7+)
- ✓ Szybki dostęp O(1)
- ✓ Heterogeniczny

Dostęp do elementów słownika

`d[key]` vs `d.get(key)`

```
d = {"name": "Alice", "age": 30}
```

Nawiasy kwadratowe

```
d["name"]      # "Alice"
```

```
d["email"]     # KeyError!
```

Metoda `get()` - bezpieczna

```
d.get("name")  # "Alice"
```

```
d.get("email") # None
```

```
d.get("email", "brak") # "brak"
```

`setdefault()` i operator `in`

Sprawdzenie klucza

```
"name" in d      # True
```

```
"email" in d     # False
```

```
"email" not in d # True
```

`setdefault` - ustaw jeśli brak

```
d.setdefault("city", "NY")
```

```
# d = {"name": "Alice",  
#      "age": 30, "city": "NY"}
```

Short-circuit z `and`

```
"name" in d and d["name"]
```



Pro tip: EAFP (Easier to Ask Forgiveness than Permission) **vs** **LBYL** (Look Before You Leap)

Pythonowy styl to EAFP (Easier to Ask Forgiveness than Permission) - używaj `try/except` zamiast sprawdzania z góry czy klucz istnieje.

Alternatywnie: `d.get(key, default)`.

Metody słownika

Metoda / operator	Opis
<code>d[key] = val</code>	Dodaje/aktualizuje parę klucz-wartość
<code>del d[key]</code>	Usuwa parę (KeyError jeśli brak)
<code>d.update(other)</code>	Łączy/aktualizuje z innym dict
<code>d.pop(key, def)</code>	Usuwa i zwraca wartość klucza
<code>d.popitem()</code>	Usuwa i zwraca ostatnią parę (klucz, wartość)
<code>d.keys()</code>	Widok kluczy (dynamiczny)
<code>d.values()</code>	Widok wartości
<code>d.items()</code>	Widok par (klucz, wartość)
<code>d other</code>	Merge słowników (Python 3.9+)
<code>d = other</code>	Merge in-place (Python 3.9+)

Widoki `keys()/values()/items()` są dynamiczne - aktualizują się gdy zmienia się słownik.

Iteracja po słowniku

Trzy widoki: klucze, wartości i pary.

```
student = {"name": "Alice", "age": 22, "grade": "A"}

# Po kluczach (domyślne)
for key in student:
    print(key)                # name, age, grade

# Po wartościach
for val in student.values():
    print(val)                # Alice, 22, A

# Po parach (najczęstsze!)
for key, val in student.items():
    print(f"{key}: {val}")    # name: Alice, age: 22, grade: A

# Elegancki sposób na ładne wypisanie
print("\n".join(f"{k}: {v}" for k, v in student.items()))
```

Dict comprehension i merge

Złożenia słownikowe i operatory łączenia.

Dict comprehension

```
# {klucz: wartosc for x in iter}  
{x: x**2 for x in range(6)}  
# {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}
```

```
# Z filtrem  
{k: v for k, v in d.items()  
 if v > 0}
```

```
# Zamiana klucz <-> wartosc  
{v: k for k, v in d.items()}
```

Łączenie słowników

```
a = {"x": 1, "y": 2}  
b = {"y": 3, "z": 4}
```

```
# Python 3.9+ - operator |  
merged = a | b  
# {"x":1, "y":3, "z":4}
```

```
# Starsza wersja  
merged = {**a, **b}
```

```
# In-place  
a |= b # lub a.update(b)
```



collections.defaultdict — słownik z domyślną wartością

```
from collections import defaultdict  
counts = defaultdict(int) # domyślnie 0  
for ch in "abracadabra": counts[ch] += 1 # {"a":5, "b":2, "r":2, "c":1, "d":1}
```

collections.Counter - sprytne zliczanie

Specjalizowany słownik do zliczania wystąpień elementów.

```
from collections import Counter

# Zliczanie znaków
c = Counter("abracadabra")
print(c)  # Counter({'a':5, 'b':2, 'r':2, 'c':1, 'd':1})

# Najczęstsze elementy
c.most_common(2)  # [('a', 5), ('b', 2)]

# Zliczanie słów w tekście
words = "the cat sat on the mat the cat".split()
word_count = Counter(words)
print(word_count)  # Counter({'the':3, 'cat':2, 'sat':1, 'on':1, 'mat':1})

# Operacje na Counterach
c1 = Counter(a=3, b=1)
c2 = Counter(a=1, b=2)
c1 + c2  # Counter({'a':4, 'b':3})
c1 - c2  # Counter({'a':2}) - ujemne znikają!
```

set i frozenset

Zbiory - unikalność i operacje matematyczne

Tworzenie zbiorów

Nieuporządkowana kolekcja unikatowych elementów.

Tworzenie

```
# Klamry (ale NIE puste!)
s1 = {1, 2, 3, 2, 1} # {1, 2, 3}

# Konstruktor set()
s2 = set("abracadabra")
# {'a', 'b', 'r', 'c', 'd'}

s3 = set(range(5)) # {0,1,2,3,4}

# UWAGA: {} to pusty DICT!
empty_set = set() # poprawnie
```

Podstawowe metody

```
s = {1, 2, 3}

s.add(4) # {1, 2, 3, 4}
s.remove(2) # {1, 3, 4}
# KeyError jeśli brak!

s.discard(99) # nic się nie stanie
s.pop() # losowy element
s.clear() # pusty set

5 in s # True/False
```

```
# Set comprehension: {wyrażenie for x in iter if warunek}
even = {x for x in range(20) if x % 2 == 0} # {0, 2, 4, 6, 8, 10, 12, 14, 16, 18}
```

Operacje na zbiorach

Znane z matematyki - unia, przecięcie, różnica, różnica symetryczna.

Operacja	Operator	Metoda	Opis
Unia (suma)	$s1 \mid s2$	<code>s1.union(s2)</code>	Wszystkie elementy z obu zbiorów
Przecięcie	$s1 \& s2$	<code>s1.intersection(s2)</code>	Wspólne elementy
Różnica	$s1 - s2$	<code>s1.difference(s2)</code>	Elementy z <code>s1</code> , których nie ma w <code>s2</code>
Różn. symetr.	$s1 \wedge s2$	<code>s1.symmetric_diff...(s2)</code>	Elementy z <code>s1</code> LUB <code>s2</code> , ale nie z obu

```
s1 = {"a", "A", "z", "Z", "G", "g"}      s2 = {"b", "B", "z", "Z", "G", "k"}
s1 | s2  # {'a', 'A', 'b', 'B', 'g', 'G', 'k', 'z', 'Z'}      s1 & s2  # {'z', 'G', 'Z'}      s1 - s2  # {'a', 'A', 'g'}
```

Relacje zbiorów i zastosowania

Relacje

```
s1 = {1, 2}
s2 = {1, 2, 3, 4}

s1.issubset(s2)    # True (s1 <= s2)
s2.issuperset(s1)  # True (s2 >= s1)
s1.isdisjoint(s2)  # False (mają wspólne)
```

Praktyczne zastosowania

```
# Usuwanie duplikatów
uniq = list(set([1,2,2,3,3,3]))

# Szybkie sprawdzenie przynależności
valid = {"admin","editor","viewer"}
if role in valid: ...

# Wspólni znajomi
common = friends_a & friends_b
```

❄ frozenset - niemutowalny zbiór

```
fs = frozenset([1, 2, 3])    # Nie ma add(), remove(), pop()...
fs2 = frozenset("hello")     # frozenset({'h','e','l','o'})
# Może być kluczem dict lub elementem set!
d = {frozenset({1,2}): "pair"} # OK!    {frozenset({1,2}): "pair"}
```

Iteracja po kolekcjach

range, enumerate, zip i inne narzędzia

range() - generator sekwencji

Leniwy generator - nie tworzy listy w pamięci, produkuje elementy na żądanie.

```
# range(stop) -od 0 do stop-1
list(range(5)) # [0, 1, 2, 3, 4]
# range(start, stop) -od start do stop-1
list(range(2, 7)) # [2, 3, 4, 5, 6]
# range(start, stop, step) -z krokiem
list(range(0, 20, 3)) # [0, 3, 6, 9, 12, 15, 18]
list(range(10, 0, -2)) # [10, 8, 6, 4, 2]
# PAMIĘTAJ: range jest generatorem, nie listą!
r = range(1_000_000) # zużywa kilkadziesiąt bajtów
l = list(range(1_000_000)) # zużywa ~8 MB pamięci!
# Sprawdzanie przynależności: O(1)!
999_999 in range(1_000_000) # True, natychmiast
```

enumerate() - indeks + element

Zwraca pary (indeks, element) - idealne do pętli for.

Podstawowe użycie

```
fruits = ["apple", "banana", "cherry"]
```

```
for i, fruit in enumerate(fruits):  
    print(f"{i}: {fruit}")
```

```
# 0: apple  
# 1: banana  
# 2: cherry
```

```
# Start od innego indeksu
```

```
for i, f in enumerate(fruits, start=1):  
    print(f"{i}. {f}")
```

✗ vs ✓ antypattern

```
# ✗ NIE-pythonowe  
for i in range(len(fruits)):  
    print(i, fruits[i])
```

```
# ✓ Pythonowe  
for i, fruit in enumerate(fruits):  
    print(i, fruit)
```

```
# ✗ Jeszcze gorzej  
i = 0  
for fruit in fruits:  
    print(i, fruit)  
    i += 1
```

enumerate() zwraca obiekt iterowalny — jednorazowy! Drugie przejście daje 0 elementów.

zip() - równoległa iteracja

łączy wiele iterowalnych w pary (krotki).

Podstawowe zip()

```
names = ["Alice", "Bob", "Carol"]
ages  = [25, 30, 35]

for name, age in zip(names, ages):
    print(f"{name}: {age}")

# Tworzenie dict z dwóch list
data = dict(zip(names, ages))
# {"Alice":25, "Bob":30, "Carol":35}

# Rozpakowanie zip (unzip)
pairs = [(1,"a"),(2,"b"),(3,"c")]
nums, chars = zip(*pairs)
```

Różne długości

```
a = [1, 2, 3]
b = ["x", "y"]

# Domyślnie: ucina do krótszego
list(zip(a, b)) # [(1,'x'),(2,'y')]

# strict=True (Python 3.10+)
list(zip(a, b, strict=True))
# ValueError: not same length!

# zip_longest – wypełnia None
from itertools import zip_longest
list(zip_longest(a, b))
# [(1,'x'),(2,'y'),(3,None)]
```

Pętla for: break, continue, else

Blok else po pętli for - rzadko spotykana, ale przydatna konstrukcja.

break i continue

```
# break – przerywa pętlę
for x in range(10):
    if x == 5:
        break      # kończy pętlę
    print(x)        # 0,1,2,3,4

# continue - pomija iterację
for x in range(5):
    if x == 2:
        continue   # pomija 2
    print(x)        # 0,1,3,4
```

for ... else

```
# else wykonuje się jeśli
# pętla NIE została przerwana break

for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            break
    else:
        # nie było break = liczba pierwsza
        print(f"{n} jest pierwsza")
# 2,3,5,7 jest pierwsza
```

⚠ else przy for to nie jest "jeśli pętla się nie wykonała" — to "jeśli nie było break".

Pętla while i walrus operator

Podstawowy while

```
n = 5
while n > 0:
    print(n, end=" ")
    n -= 1
# 5 4 3 2 1

# Nieskończona z break
while True:
    line = input("> ")
    if line == "quit":
        break
```

Walrus := w while

```
# Bez walrus
line = input("> ")
while line != "quit":
    print(line)
    line = input("> ")

# Z walrus - krócej!
while (line := input("> ")) != "quit":
    print(line)
```

Typowe wzorce z while

```
# Czekanie na warunek
import time
while not ready():
    time.sleep(0.1)
```

```
# Przetwarzanie strumienia
import sys
while (chunk := sys.stdin.read(1024)):
    process(chunk)
```

Pułapki i idiomy

Typowe błędy i pythonowe wzorce z kolekcjami

Mutowalny argument domyślny

Klasyczny błąd - domyślna lista jest współdzielona między wywołaniami!

✗ BŁĄD

```
def dodaj(x, lista=[]):
    lista.append(x)
    return lista
```

```
dodaj(1) # [1]
dodaj(2) # [1, 2] ?!
dodaj(3) # [1, 2, 3] ?!
```

```
# Ta sama lista jest używana
# za każdym wywołaniem!
```

✓ POPRAWNIE

```
def dodaj(x, lista=None):
    if lista is None:
        lista = []
    lista.append(x)
    return lista
```

```
dodaj(1) # [1]
dodaj(2) # [2]
dodaj(3) # [3]
```

```
# Każde wywołanie tworzy
# nową listę.
```

⚠ Nigdy nie używaj mutowalnego obiektu (list, dict, set) jako domyślnego argumentu funkcji!

Modyfikacja kolekcji podczas iteracji

Dodawanie/usuwanie elementów w trakcie pętli for - przepis na katastrofę.

✗ Niebezpieczne

```
nums = [1, 2, 3, 4, 5, 6]

# NIGDY tak nie rób!
for x in nums:
    if x % 2 == 0:
        nums.remove(x)
# Wynik: [1, 3, 5, 6]
# - 6 nie zostało usunięte!
```

✓ Bezpieczne metody

```
# 1. Nowa lista (comprehension)
odds = [x for x in nums if x%2 != 0]

# 2. Kopia listy
for x in nums[:]: # kopia!
    if x % 2 == 0:
        nums.remove(x)

# 3. Iteracja od tyłu
for i in range(len(nums)-1, -1, -1):
    if nums[i] % 2 == 0:
        del nums[i]
```

Pułapki ze słownikami

KeyError przy dostępie

```
d = {"a": 1, "b": 2}
```

```
# ❌ Może rzucić KeyError
val = d["c"]
```

```
# ✓ Bezpiecznie
val = d.get("c", 0)
```

True == 1 jako klucz!

```
d = {True: "prawda", 1: "jeden"}
print(d)  # {True: "jeden"}
# True == 1 i hash(True)==hash(1)
# Więc Python traktuje je
# jako TEN SAM klucz!
len(d)    # 1, nie 2!
```

Kolejność kluczy w dict

```
# Od Python 3.7+: dict zachowuje kolejność wstawiania
d = {"b": 2, "a": 1, "c": 3}
list(d.keys())  # ["b", "a", "c"] - kolejność zachowana

# Sortowanie słownika po kluczach
sorted_d = dict(sorted(d.items()))  # {"a":1, "b":2, "c":3}
# Po wartościach
sorted_v = dict(sorted(d.items(), key=lambda x: x[1]))
```

Wydajność operatora in

Wybór struktury danych ma ogromny wpływ na szybkość wyszukiwania.

Struktura	x in kolekcja	Złożoność	1M elementów
tuple	Sekwencyjne przeszukiwanie	$O(n)$	~50 ms
	Sekwencyjne przeszukiwanie	$O(n)$	~50 ms
dict (klucze)	Lookup po hashu	$O(1)$	~0.0001 ms
	Lookup po hashu	$O(1)$	~0.0001 ms

```
# Jeśli sprawdzasz przynależność wielokrotnie - zamień listę na set!
blacklist_list = ["spam@x.com", "hack@y.com", ...] # 100k elementów
blacklist_set = set(blacklist_list)                # jednorazowa konwersja

email in blacklist_list #  $O(n)$  - wolne przy każdym sprawdzeniu
email in blacklist_set  #  $O(1)$  - błyskawiczne!
```

Wybór kolekcji - ściągą

Cecha	list	tuple	dict	set
Mutowalny	✓	✗	✓	✓
Uporządkowany	✓	✓	✓ (3.7+)	✗
Indeksowany	✓ [i]	✓ [i]	✓ [key]	✗
Duplikaty	✓	✓	klucze ✗	✗
Hashowalny	✗	✓*	✗	✗
in O(1)	✗ O(n)	✗ O(n)	✓	✓
Typowe użycie	Sekwencja	Stałe	Mapowanie	Unikaty

* krotka jest hashowalna tylko gdy wszystkie jej elementy są hashowalne

Comprehension - pełen przegląd

List, dict, set comprehension + generator expression.

List	<pre>[x**2 for x in range(5)]</pre> → [0, 1, 4, 9, 16]	list
Dict	<pre>{x: x**2 for x in range(5)}</pre> → {0:0, 1:1, 2:4, ...}	dict
Set	<pre>{x%3 for x in range(10)}</pre> → {0, 1, 2}	set
Generator	<pre>(x**2 for x in range(5))</pre> → <generator object>	leniwy!

💡 Generator expression zużywa pamięć $O(1)$ - idealne do przetwarzania dużych danych: `sum(x**2 for x in range(10**6))`

Przydatne funkcje wbudowane

Funkcje, które działają z każdą kolekcją.

Funkcja	Opis	Przykład
<code>len(x)</code>	Liczba elementów	<code>len([1,2,3])</code> # 3
<code>sum(x)</code>	Suma elementów (liczby)	<code>sum([1,2,3])</code> # 6
<code>min(x) / max(x)</code>	Minimum / maksimum	<code>min([3,1,2])</code> # 1
<code>sorted(x)</code>	NOWA posortowana lista	<code>sorted({3,1,2})</code> # [1,2,3]
<code>reversed(x)</code>	Iterator od końca	<code>list(reversed([1,2,3]))</code> # [3,2,1]
<code>all(x)</code>	True jeśli wszystkie truthy	<code>all([1,2,3])</code> # True
<code>any(x)</code>	True jeśli chociaż jedno truthy	<code>any([0, '', 1])</code> # True
<code>list()/tuple()/set()</code>	Konwersja między kolekcjami	<code>set([1,1,2,3])</code> # {1,2,3}

Rozpakowywanie słowników

```
# Rozpakowywanie do zmiennych (po kluczach)
def greet(name, age):
    print(f"Cześć, {name}! Masz {age} lat.")

data = {"name": "Alice", "age": 25}
greet(**data) # greet(name="Alice", age=25)

# Łączenie słowników (Python 3.5+)
defaults = {"color": "blue", "size": "M", "qty": 1}
custom = {"size": "XL", "qty": 3}

result = {**defaults, **custom}
# {"color": "blue", "size": "XL", "qty": 3} - custom nadpisuje

# To samo co result = defaults | custom (Python 3.9+)
```

itertools - zaawansowana iteracja

Moduł biblioteki standardowej z potężnymi narzędziami iteracyjnymi.

```
from itertools import chain, product, combinations, permutations, groupby
```

```
# chain - łączenie iterowalnych
```

```
list(chain([1,2], [3,4], [5])) # [1, 2, 3, 4, 5]
```

```
# product - iloczyn kartezjański
```

```
list(product("AB", [1,2])) # [('A',1),('A',2),('B',1),('B',2)]
```

```
# combinations - kombinacje
```

```
list(combinations("ABCD", 2)) # [('A','B'),('A','C'),...,( 'C','D')]
```

```
# permutations - permutacje
```

```
list(permutations("ABC", 2)) # [('A','B'),('A','C'),...,( 'C','B')]
```

```
# groupby - grupowanie (dane muszą być posortowane!)
```

```
data = [("A",1),("A",2),("B",3)]
```

```
{k: list(g) for k, g in groupby(data, key=lambda x: x[0])}
```

Triki i ciekawostki

◆ Transponowanie macierzy

```
matrix = [[1,2],[3,4],[5,6]]  
list(zip(*matrix)) # [(1,3,5),(2,4,6)]
```

◆ Spłaszczenie listy

```
flat = [x for sub in nested for x in sub]
```

◆ Najczęstszy element

```
max(set(data), key=data.count)  
# lub Counter(data).most_common(1)
```

◆ Odwrócenie słownika

```
{v:k for k,v in d.items()}
```

◆ Lista unikatów z zachowaniem kolejności

```
list(dict.fromkeys([3,1,2,1,3]))  
# [3, 1, 2] - dict zachowuje kolejność!
```

Pamięć: rozmiary kolekcji

`sys.getsizeof()` - ile pamięci zużywa każda kolekcja?

Typ	Pusta	10 elem.	Uwagi
	56 B	136 B	Dynamiczna tablica wskaźników
<code>tuple</code>	40 B	120 B	Stała - mniejsza od list
	64 B	352 B	Tablica hashowa
<code>set</code>	216 B	728 B	Tablica hashowa (większa niż dict)
	49 B	59 B*	* dla 10 znaków ASCII



Kluczowy wniosek

`tuple` jest lżejsze od `list`. Dla danych stałych (np. konfiguracja, stałe współrzędne) używajmy krotek. `set`/`dict` są cięższe, ale dają $O(1)$ lookup - zysk przy częstym wyszukiwaniu.

Kolekcje w Pythonie - kluczowe punkty

1 list - mutowalna sekwencja, append/pop/slicing, list comprehension

2 tuple - niemutowalna, hashowalna*, lekka, idealna do stałych danych

3 dict - mapowanie klucz→wartość, O(1) lookup, dict comprehension

4 set - unikalne elementy, operacje matematyczne, O(1) in

5 frozenset - niemutowalny set, może być kluczem dict

6 Comprehension: [list], {dict}, {set}, (generator)

7 enumerate, zip, range - idiomatyczna iteracja

8 Pułapki: mutowalny default, modyfikacja w pętli, płytka kopia